

AI Audit Report — HW03 Task 1: GUI Checklist

Mandatory appendix for every AI-assisted homework (HW#01–HW#06, and Seminar).

Adapted from Med Kharbach, PhD (2026) — AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0. This adaptation is prepared for FIT@HCMUS – CS423 / CSC15003 Software Testing course.

1. Student Information

Field	Value
Student name (printed):	DANG TRUONG NGUYEN
Student ID:	23127438
Class / Cohort:	23KTPM3
Assignment ID (e.g., HW#00, HW#02):	HW03 — Task 1 (GUI Checklist)
Assignment date:	25/07/2026
AI tool(s) used:	Claude Code (Claude Opus 4.8)
AI tool(s) used:	☒ Yes ☐ No

2. Instructions (read before filling)

- Add one row per AI-generated artifact (test case, script, checklist, OpenAPI spec, JMeter plan, etc.).
- Paste the verbatim prompt — DO NOT paraphrase.
- Paste the verbatim AI output (or include a labelled screenshot in the report).
- Tag the verdict: VALID / INVALID / INCOMPLETE.
- Reasoning must cite a course slide, ISTQB section, or technical RFC.
- Show the corrected artifact with the change highlighted.
- Sample rows are in italic — replace them before submission.

3. Audit Table — one row per artifact

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
Tool: Claude Code (Claude Opus 4.8) Artifact: GĐ1 · UI Inventory Prompt (verbatim from PLAN.md, Prompt #1): ROLE: You are helping build a UI element inventory for GUI testing (not generating checklist items yet). You have read access to the eshop-sut repository, already cloned locally. CONTEXT: EShop is a Vietnamese e-commerce demo app. Screen in scope: "<<screen name, e.g. Đăng ký / Register>>", corresponding to FR-<<OX>> in the SUT spec. The relevant code lives under frontend-web/ (React + Vite + Tailwind). The choosed screen is: STEPS: 1. Search frontend-web/src/ for the file(s)	9 UI-inventory files in reports/23127438/ui-inventory/ (register, login, forgot-password, home, product-detail, cart, checkout, order-history + _shared-layout), ~116 element rows each with Source file:line . Backend was run to cross-check API-driven content (5 seed products; search ' returns raw SQL-error HTML; /product/999 returns {}).	INCOMPLETE	ISTQB FL – test analysis derives test conditions from the test basis; a UI element inventory is that test-condition-identification step and must be traceable to the source. The static inventory was not complete until confirmed by dynamic execution (running the app).	I verified each file:line against the React source, extracted the shared header/footer into _shared-layout.md to avoid cross-screen duplication, and confirmed the runtime-only behaviours (SQL-error leak, empty product) on the running SUT before accepting.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
implementing this screen (by route, page/component name, or FR-related keyword). List every file path you used — don't guess without checking. 2. Read the full component tree: the page component + any imported sub-components, shared form/button/toast/dialog components, and every conditional-rendering branch (loading, error, empty, success, disabled states). 3. If you can run shell commands: start the dev server (<code>npm run dev</code> inside <code>frontend-web/</code> , default port 5173) and fetch/render the page to cross-check the static-code inventory against what actually renders — flag anything that only shows up at runtime. If you cannot execute or browse, skip this and say so explicitly instead of guessing. 4. Build an exhaustive UI				

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix						
element inventory for THIS SCREEN ONLY. OUTPUT: write the result to <code>reports/23127438/ui-inventory/&lt;&l</code> <code>t;screen-slug&gt;&gt;.md</code> as a markdown table: <table><tr><th>#</th><th>Element type</th><th>Element label/name</th><th>Location</th><th>Apparent purpose / linked requirement</th><th>Source file:line</th></tr></table> Include every button, field, label, link, icon, toast/dialog trigger, and any conditionally-rendered element — even ones not visible by default.	#	Element type	Element label/name	Location	Apparent purpose / linked requirement	Source file:line				
#	Element type	Element label/name	Location	Apparent purpose / linked requirement	Source file:line					

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
Tool: Claude Code (Claude Opus 4.8) Artifact: GĐ2a · Checklist IA-01 Prompt (verbatim from PLAN.md, Prompt #2a): ROLE: You are a senior QA engineer creating a requirements-based GUI checklist. CONTEXT — System Under Test: - App: EShop, Vietnamese e-commerce demo. Frontend Web (customer) at localhost:5173, React + Vite + Tailwind. - UI element inventory for the screens in scope: <<paste the combined inventory tables from Phase 1, all screens>> - Official "General UI Standards" requirement (FR-21), verbatim from the SUT repo README, section 8: <<paste FR-21 content here>> TASK: Generate GUI checklist items for IA-01 –	16 IA-01 items (ia-01-general-ui.md) – language, button colour, currency đ , single- &lt;h1&gt; , tab order, plus best-practice title/responsive items; each an objective Pass/Fail statement traced to an element + FR-21 rule or heuristic.	INCOMPLETE	ISTQB FL – requirements-based test design: every test condition must trace to the test basis. FR-21 was pasted verbatim so items map 1-to-1 to the standard; the heuristic items beyond FR-21 required human judgement to confirm relevance to this app.	I supplied FR-21 verbatim from README §8, checked each item traces to a real element, and later merged the tab-order item with IA-02 during consolidation.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
General UI standards ONLY (visual consistency, layout/alignment, typography, spacing, color usage, iconography, responsive/viewport behavior, page-title correctness). RULES: 1. Each item = one objective, testable Pass/Fail statement. 2. Each item must reference a specific screen/element from the inventory — no generic filler. 3. Include items that verify each rule in FR-21, PLUS extra best-practice items beyond FR-21 that are still specific to this app. 4. If the same rule applies identically to multiple screens, write ONE item and list all applicable screens. 5. Produce at least 12 items. OUTPUT — a markdown table: ID Screen(s) Checklist Item				

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
Expected Result Traced to (FR-21 / heuristic) . Use IDs GUI-IA01-01, GUI-IA01-02, ...				

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
Tool: Claude Code (Claude Opus 4.8) Artifact: GĐ2b · Checklist IA-02 Prompt (verbatim from PLAN.md, Prompt #2b): [Same ROLE/CONTEXT block as Prompt #2a, but swap the requirement paste-in for FR-22:] - Official "Form Requirements" (FR-22), verbatim from repo README section 8: <<paste FR-22 content here>> TASK: Generate checklist items for IA-02 – Forms ONLY (required-field indicators, input types/masking, validation timing and message placement, multi-step indicators, tab order within forms, confirmation-field matching, format constraints e.g. phone/quantity/C SV). Pay special attention to any rule in FR-22 that contradicts common UI	16 IA-02 items (ia-02- forms.md) – required * , email/password input types, error placement, step indicator, phone/password/OTP/quantity constraints, editable-total, confirm-password, tab order.	INCOMPLETE	ISTQB FL – tests must be designed against the specification, not the tester's assumptions. FR-22's error-above-submit rule is anti-convention; the prompt explicitly told the AI NOT to 'correct' it, guarding the known pitfall of testing against expectation instead of the spec.	I confirmed the AI kept the anti-convention rule (GUI-IA02-04 verifies error ABOVE the submit button, per FR-22, not the common below-field pattern) and added the missing HTML5-native validation-language item.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
conventions (e.g. unusual error-message placement) — do not silently "correct" it to the common pattern; check what THIS spec requires. [Same RULES 1–5 and OUTPUT format as #2a, IDs GUI-IA02-xx]				

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
Tool: Claude Code (Claude Opus 4.8) Artifact: GĐ2c · Checklist IA-03 Prompt (verbatim from PLAN.md, Prompt #2c): [Same ROLE/CONTEXT block, swap in FR-23:] - Official "Navigation Requirements" (FR-23), verbatim from repo README section 8: <<paste FR-23 content here>> TASK: Generate checklist items for IA-03 – Navigation ONLY (active-state highlighting, badges/counters, exact label wording, breadcrumbs on sub-pages, back/continue links, browser back-button behavior, invalid-URL/404 handling, pagination if present). [Same RULES 1–5 and OUTPUT format as #2a, IDs GUI-IA03-xx]	15 IA-03 items (ia-03- navigation.md) – active highlight, cart badge, exact 'Đăng xuất' wording, breadcrumb, 404/not-found, SPA nav, back-button, route guard, pagination.	INCOMPLETE	ISTQB FL – requirements-based design from FR-23 plus experience-based heuristics for navigation behaviours (404, back-button) not covered by the FR; the experience-based additions needed human validation for applicability.	I supplied FR-23 verbatim, kept the exact-wording checks, and confirmed the heuristic items (404, route guard) are objective Pass/Fail statements.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
Tool: Claude Code (Claude Opus 4.8) Artifact: GĐ2d · Checklist IA-04 Prompt (verbatim from PLAN.md, Prompt #2d): [Same ROLE/CONTEXT block, swap in FR-24:] - Official "Feedback & State Requirements" (FR-24), verbatim from repo README section 8: <<paste FR-24 content here>> TASK: Generate checklist items for IA-04 – Feedback/state ONLY (action feedback e.g. add-to-cart, confirmation dialogs before destructive actions, empty-state visuals, loading indicators, error/success toasts, account-lockout messaging, coupon apply/invalid feedback, image alt-text, safe rendering of user-supplied text such as	18 IA-04 items (ia-04-feedback-state.md) – add-to-cart feedback, confirmation dialogs, empty/loading/error states, alt text, lockout messaging, coupon feedback, safe rendering of user text, cart-state consistency.	INCOMPLETE	ISTQB FL – requirements-based design from FR-24 plus security-relevant checks (safe rendering of user input) drawn from the SUT README; the safe-rendering and lockout items required human confirmation against runtime behaviour.	I supplied FR-24 verbatim and, during execution, confirmed the safe-rendering item (GUI-IA04-13) is a real XSS on the running app, raising its severity to Blocker.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
search terms). [Same RULES 1–5 and OUTPUT format as #2a, IDs GUI-IA04-xx]				

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
Tool: Claude Code (Claude Opus 4.8) Artifact: GĐ3 · Gap analysis Prompt (verbatim from PLAN.md, Prompt #3): Here is my consolidated GUI checklist for the EShop app so far (<<X>> items, IA-01 to IA-04): <<paste the checklist from Phase 2>> TASK: For EACH of the following 8 dimensions, tell me whether it is already represented, partially represented, or completely absent in my checklist above. Do NOT add new checklist rows — just diagnose gaps. 1. Accessibility (keyboard-only operation beyond simple tab order, screen-reader/ARIA labels, color-contrast ratios, visible focus states) 2. Dark mode / theme switching 3. Right-to-left (RTL) layout	Diagnosis across the 8 dimensions: 0 fully / 5 partially (accessibility, extreme viewport, network, text-overflow, browser-quirks) / 3 absent (dark mode, RTL, print), with candidate items I was told to verify myself. Saved as gap-analysis.md (Part A).	INCOMPLETE	ISTQB FL – experience-based test techniques: checklists have inherent coverage gaps that exploratory/error-guessing must complement. By design the AI only diagnoses; deciding which gaps are real and WHY the AI missed them is the human analysis Task 1 §6 grades.	I ran the app myself and added 4 self-verified GUI-GAP items with root-cause reasons: cart lost on refresh, duplicate cart lines, <code>html lang=en</code> (WCAG 3.1.1), and labels without <code>htmlFor</code> (WCAG 1.3.1/4.1.2) — the last two the AI missed despite proposing the accessibility dimension itself.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
support 4. Extreme viewport sizes (narrow mobile widths, 200%+ browser zoom) 5. Network resilience (slow connection, request timeout, offline state) 6. Text-overflow / localization edge cases (very long names, number-formatting edge cases) 7. Print / export-friendly view 8. Browser-native interaction quirks (autofill vs. custom validation, back/forward cache showing stale cart state) For each dimension marked "partially represented" or "absent," suggest 1–2 candidate items — but do NOT assume they are correct or complete; I will verify against the running app myself.				

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
Tool: Claude Code (Claude Opus 4.8) Artifact: GĐ4 · Consolidated checklist Prompt (verbatim from PLAN.md, Prompt #4): I ran 4 separate checklist-generation passes (IA-01 to IA-04) plus manual additions from my own gap review. Combined raw list below: <<paste all items: Phase 2 outputs (2a+2b+2c+2d) + Phase 3 manual GUI-GAP additions>> TASK: 1. Remove exact or near-duplicate items (if two items test the same underlying rule on different screens, merge into one row and list all applicable screens). 2. Renumber sequentially within each Interface Aspect (GUI-IA01-01, ... GUI-IA04-xx). Keep manually-added items prefixed GUI-GAP-xx (do NOT renumber them	Deduped 65 AI items + 4 GUI-GAP into 66 final items (checklist-final.md); 3 near-duplicate merges (tab order, error placement, safe rendering), renumbered per aspect, GUI-GAP kept prefixed, total confirmed above 40.	INCOMPLETE	ISTQB FL – test case quality & maintainability: remove redundant tests and keep each case an objective, verifiable Pass/Fail statement. The merges changed IDs, so traceability had to be re-verified by a human.	I reviewed every merge, kept a dedup log for traceability, later removed the Source column on request, and confirmed the per-aspect counts (17/15/15/19 = 66).

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
into the IA sequence) so it stays traceable which items were AI-generated vs. manually added. 3. Verify each item is an objective, testable Pass/Fail statement — rewrite any vague/subjective one. 4. Output the final table: ID Interface Aspect Screen(s) Checklist Item Expected Result Source (AI-generated / Manually added) 5. At the end, report total item count and per-aspect breakdown, and confirm the total is above 40.				

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
Tool: Claude Code (Claude Opus 4.8) Artifact: GĐ6 · Bug reports / GitHub issues Prompt (verbatim from PLAN.md, Prompt #6): I found a UI bug while executing GUI checklist item <<ID>> on the EShop app. Raw details (all provided by me — do not add anything I have not stated): - Screen: <<...>> - Steps I performed: <<...>> - Expected result (per checklist / FR-XX): <<...>> - Actual result observed: <<...>> - Environment: <<browser/OS/device>> TASK: Draft a GitHub issue using exactly this structure: ## Title / ## Description / ## Steps to Reproduce / ## Expected Result / ## Actual Result / ## Environment / ## Related checklist item / ## Severity [Blocker / Major /	GitHub-issue drafts for the 57 failed items, grouped by root cause into 48 bugs (bug-report.md + issues/BUG-01..48.md); each with title/description/steps/expected/actual/severity/checklist link. Filed as issues #194–241 with Cloudinary screenshots.	INCOMPLETE	ISTQB FL – defect management: a defect report must be traceable, reproducible, and carry actual-vs-expected results. The AI produced issue-ready structure, but severity ratings and screenshot evidence required human confirmation before filing.	I grouped 57 fails into 48 distinct defects, confirmed each severity, attached the real screenshot per bug, and back-filled the issue number into every test case for two-way traceability.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
Minor / Cosmetic, with 1-sentence justification] / ## Screenshot. Do not invent, assume, or embellish any detail beyond what I gave you above.				

4. Summary of AI Accuracy

Aggregate the verdicts from Section 3 and complete the table below.

Metric	Count	Percentage
Total AI-generated artifacts audited	8	100%
VALID (correct, accepted as-is)	0	0%
INVALID (wrong; rejected)	0	0%
INCOMPLETE (acceptable after edits)	8	100%

5. Conclusion — When should AI be used (or not)?

Write 80–150 words describing patterns you observed. Where did AI shine? Where did AI fail? What is your recommendation for using AI in this kind of work in the future?

Instead of relying on AI to generate test cases directly, I used Claude to streamline the testing workflow. Claude was first used to analyze the UI inventory and extract it into a structured draft checklist. I then manually reviewed the checklist, identified missing scenarios and edge cases that the AI did not capture, and added these gaps before executing the tests manually.

During the testing process, Claude also assisted with formatting issue reports and preparing them for submission to GitHub. While this significantly reduced the time spent on documentation and organization, the quality of the final results still depended on manual validation and domain knowledge. AI proved to be an effective productivity tool for generating drafts and improving consistency, but human judgment remained essential for identifying missing test scenarios, verifying defects, and ensuring the final deliverables met the assignment requirements.

6. Mandatory Disclosure (paste verbatim)

UI Inventory, checklist draft was initially generated by Claude; I reviewed and confirmed, added 4 gap GUI; section 5 was written entirely by me. The detailed AI Audit Report is attached as Appendix A. I confirm I did not use AI to generate any artifact listed in the prohibited category.

Signature

Student name (printed):	DANG TRUONG NGUYEN
Student ID:	23127438
Class / Cohort:	23KTPM3
Course:	CS423 / CSC13003 – Software Testing
Instructor:	Msc. Tran Thi Bich Hanh
Date:	25/07/2026
Signature:	

References

- Kharbach, M. (2026). AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0.
- ISTQB Foundation Level Syllabus (latest version).
- Hardman, P. (2025). A Post-AI Learning Taxonomy.
- Fuster Rabella, M. (2025). OECD Education Working Paper No. 338.
- Perkins, M., Roe, J., & Furze, L. (2025). AI Assessment Scale.
- Anthropic (2025). Building reliable AI test agents — engineering blog.
- DeepEval & Promptfoo documentation — testing frameworks for LLM systems.